Aggregations

The core aspect of metrics is the ability to perform aggregations over time on a grouping entity (or a set of grouping entities). The general rule when creating the actual metrics is always to keep it as simple as possible. This applies also to the window definition and to the actual aggregation expression. Let's break down each one of these:

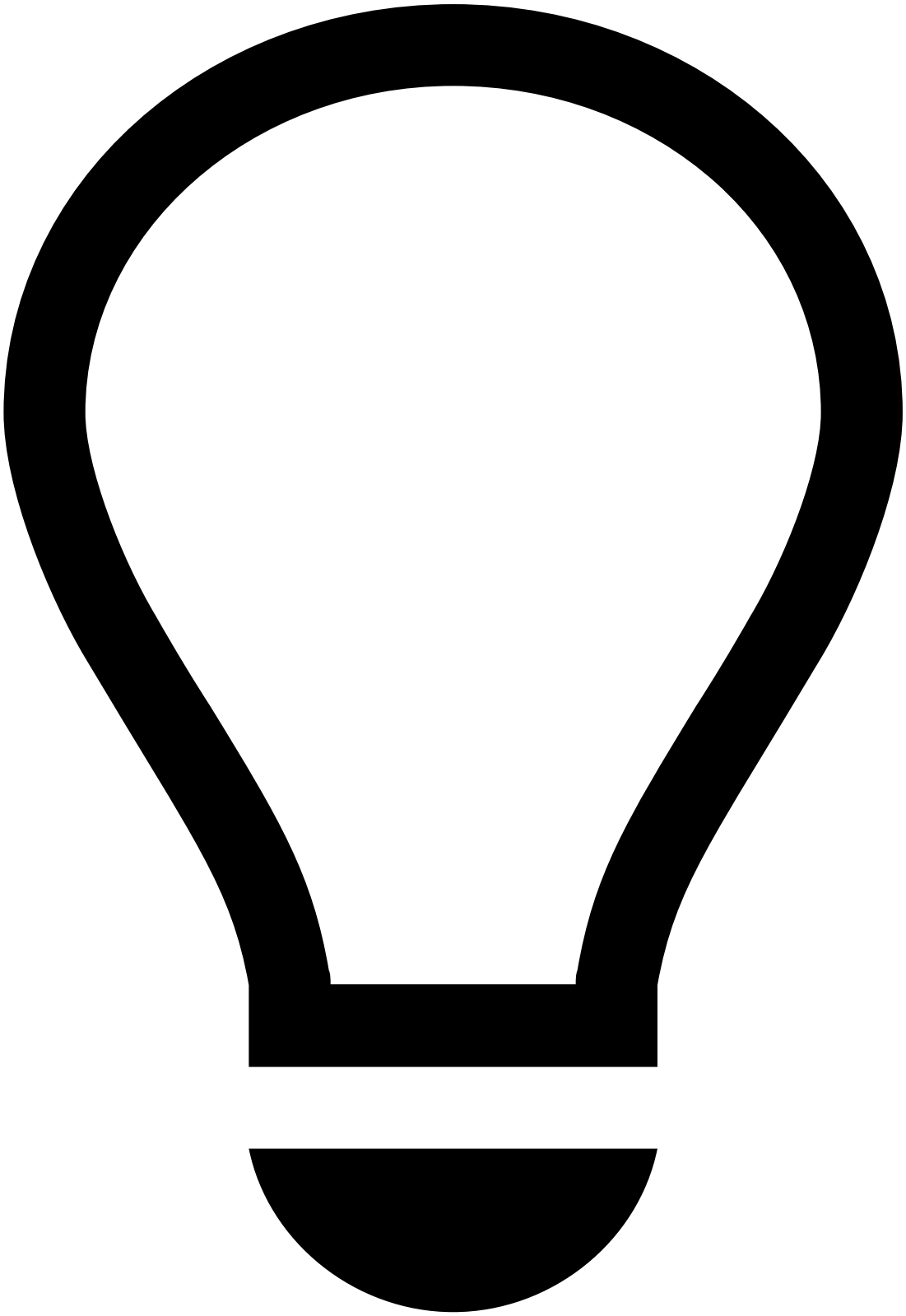
Windowsâ□□

The rule of thumb is to have two main sets of windows: 1 for real time (e.g. 24 hours), and another for scheduled metrics (e.g. 1 month). However, note that with very large payloads in runtime, the size of the real time windows might need to be adjusted. Real time windows, albeit faster to compute, imply that data that goes inside the window (c.f. filters) must be

kept in memory. Also, during recovery, the data pertaining to the largest real-time window must be reinjected into the workflow before operations can resume.

Group-byâ□□

When using the `Group By` operator to group by multiple fields, always have the high-cardinality ones first since they will dictate how the data is partitioned in the Spark shuffle (if you want to group by `card_pan` and `merchant_id`, and there are 1M cards but just 100 merchants, you will want to select `card_pan` first to avoid having 100 very large tasks that might run out of memory).



tip

This manual optimization is not necessary when using Railgun.

Aggregationâ□□

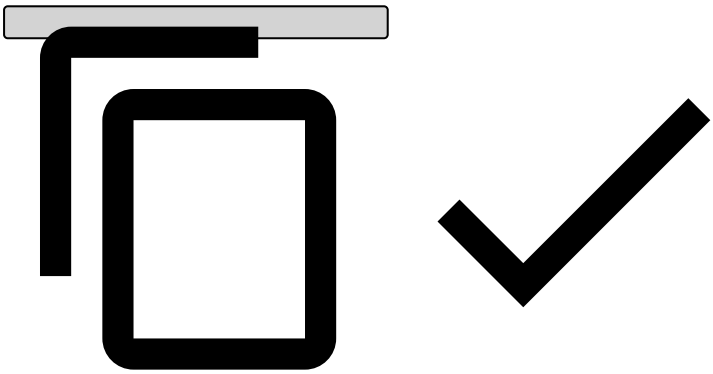
The "aggregation" input lets you define what type of aggregation is to be done. Note that the `countDistinct` aggregation is more performant in runtime than in the Data Science Platform. This means that having a lot of `countDistinct` s will make your Data Science job slower. In addition, these are memory intensive aggregations, as the whole state is needed to be able to expire elements as the clock moves forward.

Some advanced aggregation examples:

Velocity of transactionsâ

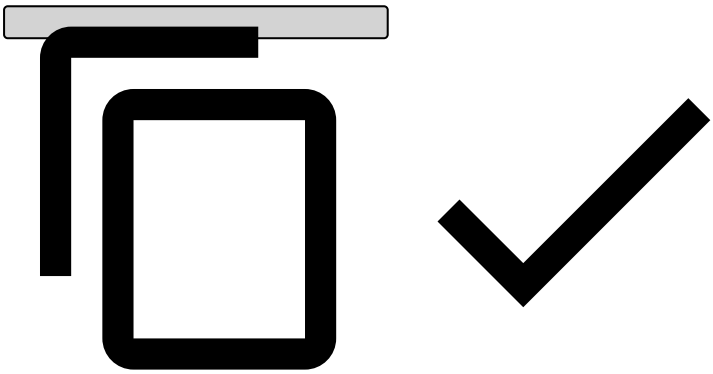
```
(
    let first_timestamp = first().event_occurred_at ?? 0 in
    let last_timestamp  = last().event_occurred_at  ?? 0 in
    let time_span       = (1 + last_timestamp - first_timestamp ) / ( 60*60*1000.0 ) in
    let count           = count() in

    ( if ( count <= 1 ) then 0 else count / time_span )
)
```



The time since the previous event within this aggregationâ

```
// (in milliseconds)
(
    ( last().event_occurred_at ?? 1 ) - ( prev().event_occurred_at ?? 1 )
)
```



Dosâ

- Use "simple" aggregations (from the UI).
- Use Approximate count distincts if you expect the counts to be large.
- Create an "aggregation repository" with the expressions that are commonly used. This will help you avoid bugs.

Don'tsâ

- Avoid writing too many advanced aggregations.
- Avoid count distincts with long windows. Distinct aggregations are the most costly, because all the events must be kept inside the window at all times. (c.f. Approximate aggregations). This cost mostly affects the Data Science Platform and not necessarily Runtime (see Scheduled Metrics Jobs).
- Don't use real-time metrics with long windows unless you have direct support from Feedzai's product team. The experimental nature of the feature means that it is not recommended for general usage.

Advanced optimization techniquesâ□□

- Salting null values/large entities that can be ignored. SMJs already ignore null values.
- Metrics are very sensitive to data skew and suboptimal data partitioning. Make sure your data sources are well partitioned by checking that each uncompressed CSV part takes about 128 MB in HDFS or closer to 512 MB in S3. That being said, as a rule of thumb, as long as each part is about 100 MB in size, it should be good enough. If you have too many partitions, you can coalesce them manually. Check the Spark documentation for more information on coalescing.
- Fields that are not necessary in a feature engineering plan should be dropped at the beginning, or in a preprocessing plan or JupyterLab script. This is due to Spark that sends data across machines during execution. By removing columns that are unused, Spark will send less data between machines. This will allow you to use less memory per executor, thus providing more parallelism. **Rule of thumb**: start by looking at large complex fields and constant fields.
- Filters inside metrics do provide *some* improvements in memory usage. They are not as impactful as removing unnecessary fields. Generally speaking, if you are measuring something that can be tracked only in specific event types, try to have as specific a filter as possible. Even though this won't reduce the bulk memory consumption by Spark, it will reduce memory being consumed during the aggregation.